Incremental Predictive Value of Code Smells Beyond Code, Development History, and Commit Changes for Fault-Inducing Commit Prediction

ABHINAV J
School of Computer Science and Engineering (SCOPE)
VIT-AP University
Amaravati, India
abhinavjammalamadaka@gmail.com

BALAJI M
School of Computer Science and Engineering (SCOPE)
VIT-AP University
Amaravati, India
mannepallibalaji9@gmail.com

Durga Sai K
School of Computer Science and Engineering (SCOPE)
VIT-AP University
Amaravati, India
karakambadidurgasai123@gmail.com

Abhishek A
School of Computer Science and Engineering (SCOPE)
VIT-AP University
Amaravati, India
ajameraabhi@gmail.com

***Abstract*—Code smells have been investigated as indicators of fault-prone software, but their incremental predictive value beyond conventional code and development-history characteristics remains less clear when current commit changes are also considered. This study investigates whether pre-commit code-smell information provides additional predictive value for identifying SZZ-derived fault-inducing commits. We use Version 2.0 of the Technical Debt Dataset and construct a final modeling population of 66,531 commits from 31 projects. Four nested feature configurations are evaluated: conventional code metrics (C), code metrics with development history (C+H), the addition of current commit-change characteristics (C+H+Change), and the further addition of pre-commit code-smell features (C+H+Change+S). Three classifiers are evaluated using chronologically ordered data, supplemented by ten rolling temporal windows and leave-one-project-out validation. In the primary temporal evaluation, adding code-smell features increases PR-AUC by 0.0067–0.0095 across the three classifiers. In comparison, adding commit-change characteristics increases PR-AUC by 0.1508–0.1757 over the C+H baseline. Across rolling temporal windows, the mean smell-related PR-AUC change is positive for all three classifiers, but the effect varies across windows. Leave-one-project-out evaluation similarly provides no consistent evidence that the incremental smell contribution transfers to held-out projects within the dataset. These results indicate that pre-commit code-smell information contains a small amount of additional predictive information in the evaluated temporal setting, while current commit-change characteristics provide substantially larger incremental predictive value.**

***Index Terms*—code smells, defect prediction, just-in-time software quality assurance, SZZ, technical debt, mining software repositories, temporal validation, cross-project defect prediction**

I. Introduction

Software defects remain an important concern in modern software development because identifying potentially fault-prone changes before they propagate can help development teams allocate testing and review effort more effectively. Just-in-time (JIT) defect prediction addresses this problem at the level of individual code changes, using information available around the time a change is introduced rather than relying only on aggregated project-level measurements.

A large body of JIT defect-prediction research has investigated characteristics of source-code changes and development history as predictors of future defects. Kamei *et al.* conducted a large-scale empirical study of JIT quality assurance and demonstrated the value of change-level and development-process characteristics for identifying potentially defective changes [1]. At the same time, code smells have attracted considerable attention as indicators of potentially problematic design and implementation characteristics. Empirical studies have reported relationships between particular smells and software faults, while other studies have incorporated smell information directly into defect-prediction models [2]–[4].

However, an association between code smells and faults does not establish that smell information provides substantial additional predictive information once other relevant characteristics are already available. A smell-based predictor may overlap with conventional code metrics, development history, or characteristics of the current modification. Evaluating smells independently without controlling for these feature groups can therefore make it difficult to determine their incremental predictive contribution.

Previous research has also examined smell-based cross-project defect prediction. Sotto-Mayor and Kalech studied cross-project smell-based defect prediction and compared smell information with conventional code metrics [5]. Such work demonstrates that smell information can be useful under particular cross-project protocols, but it does not by itself establish whether smells provide additional information be-

yond a combined baseline of code, history, and current change characteristics.

This study therefore frames the problem as an incremental-value question. We begin with conventional static code metrics (C), add development-history characteristics (H), then current commit-change characteristics (Change), and finally add pre-commit code-smell information (S). This nested construction allows the change associated with each feature group to be measured directly.

The study uses Version 2.0 of the Technical Debt Dataset, which integrates repository, issue-tracking, static-analysis, refactoring, and SZZ-derived information across 31 projects [8]. The target is an SZZ-derived fault-inducing-commit label rather than a manually verified causal defect label. To reduce temporal leakage, the primary evaluation is chronologically ordered. Ten rolling windows examine temporal stability, and leave-one-project-out (LOPO) evaluation examines transfer to projects excluded from model training.

The research questions are:

- RQ1: Does adding pre-commit code-smell information improve prediction of SZZ-derived fault-inducing commits beyond static code, development history, and commit-change information?
- RQ2: How does the incremental predictive contribution of code smells vary across successive temporal evaluation windows?
- RQ3: Does the incremental predictive contribution of code smells remain consistent when entire projects are excluded from model training?

The contributions are: (1) a nested feature-group design isolating smell contribution; (2) chronological and rolling temporal evaluation; (3) project-held-out evaluation; (4) an empirical comparison of the relative contribution of history, current changes, and smells; and (5) an audit of SZZ label timing and available follow-up.

II. Related Work

A. Just-In-Time Defect Prediction

JIT defect prediction evaluates individual software changes rather than only aggregate modules. Kamei *et al.* established a large-scale empirical basis for JIT quality assurance using change and development-process characteristics [1]. The present study retains the change-level unit but asks a narrower question: after code, history, and current-change information are known, how much additional information is supplied by pre-commit smells?

B. Code Smells and Faults

Hall *et al.* reported that some smells had significant relationships with faults, while effects were small and system-dependent [2]. Palomba *et al.* investigated smell intensity for bug prediction and later developed a smell-aware prediction model using smell information with conventional product and process metrics [3], [4]. These studies motivate smell-aware prediction but do not make incremental contribution the only object of evaluation.

C. Cross-Project Prediction

Sotto-Mayor and Kalech investigated cross-project smell-based defect prediction using smell and code-metric information [5]. The present LOPO protocol has a different objective: it measures whether the incremental smell contribution observed after adding the same baseline feature groups remains when an entire project is withheld.

D. SZZ and Fault-Label Construction

SZZ traces bug-fixing changes to earlier commits to identify candidate fault-inducing commits. Because the procedure is an operational labeling mechanism, its output is not equivalent to manually verified causal ground truth. Campos Neto *et al.* revisited SZZ implementations and documented implementation-related limitations [6]. We therefore use the precise term "SZZ-derived fault-inducing commit" throughout this paper and separately audit the temporal properties of the labels.

E. Research Gap

Existing work establishes that code smells can be associated with faults and can be incorporated into prediction models, and cross-project smell-based prediction has also been studied. We have not identified an exact prior study matching the specific combination evaluated here: nested C $\rightarrow$ C+H $\rightarrow$ C+H+Change $\rightarrow$ C+H+Change+S feature construction, an SZZ-derived change-level target, chronological evaluation, ten rolling temporal windows, and project-held-out evaluation of the incremental smell contribution. The contribution of this study is therefore the experimental characterization of incremental predictive value rather than the broader claim that code smells are related to defects.

III. Dataset and Data Construction

A. Technical Debt Dataset

Version 2.0 of the Technical Debt Dataset is the primary empirical source [8]. The official V2 release contains 31 projects and analyzes the master branch; its README states that V2 uses the maintainers' fork of SZZUnleashed for fault-inducing and fault-fixing links. The study uses the V2 SQLite database containing 31 projects.

The tables serve different roles. Git commits and commit changes provide the prediction unit and change features; SonarQube analyses, issues, and measures provide pre-commit static and smell information; Refactoring Miner provides supplementary repository analysis; and SZZ relationships provide the target labels.

B. Dataset Inventory and Filtering

Table I summarizes the principal source tables used during dataset construction. The counts are retained in the paper because they provide an auditable view of the raw evidence from which the modeling population was derived.

The construction process is auditable as a sequence of explicit filters. Starting from 153,994 Git commits, the builder retains commits with exactly one parent, excludes commits

TABLE I
PRINCIPAL SOURCE TABLES IN THE TECHNICAL DEBT DATASET DATABASE.

Table	Records	Role in this study
GIT_COMMITS	153,994	Commit metadata, dates, parents
Git changes	1,142,878	File-level additions/removals
SONAR_ANALYSIS	67,550	Static-analysis revisions
SONAR_ISSUES	1,024,614	Smell lifecycle information
SONAR_MEASURES	66,711	Conventional code metrics
SONAR_RULES	1,819	Rule definitions and identifiers
Refactoring Miner	362,253	Refactoring observations
JIRA_ISSUES	61,402	Issue-tracking records
SZZ fault links	52,428	SZZ-derived labels
PROJECTS	31	Project identifiers

TABLE II
CANDIDATE POPULATION AFTER SUCCESSIVE CONSTRUCTION STAGES.

Stage	Total	Positive	Negative
Initial eligible candidates	67,375	9,635	57,740
Final modeling population	66,531	9,295	57,236
Chronological training set	46,571	–	–
Chronological test set	19,960	–	–

marked as merges, requires a parent SonarQube analysis for the project/parent pair, and assigns the SZZ label from the distinct (project, inducing-commit) pairs in the SZZ table. This produces 67,375 initial eligible candidates. A downstream parent-analysis availability check used by the augmented modeling branch removes 844 observations, yielding 66,531 modeling instances. The excluded set contains 340 positives and 504 negatives; therefore 3.53% of the initial positives and 0.87% of the initial negatives are removed. We do not assume this missingness is random.

C. Candidate Construction

The unit of prediction is an individual Git commit. An initial population of 67,375 eligible commits contained 9,635 SZZ-positive instances and 57,740 negatives. A target commit was retained for modeling only when the required parent and pre-commit analysis could be established. The final modeling population contained 66,531 commits from all 31 projects: 9,295 positive and 57,236 negative instances, for a positive prevalence of 13.97%.

For target construction, the SZZ table is first reduced to distinct (project, fault-inducing-commit) pairs. A candidate commit is labeled positive when its (project, commit hash) pair occurs in this set; multiple fixing links to the same inducing commit therefore contribute one binary positive instance. The 52,428 source-table links are consequently not interpreted as 52,428 unique positive commits.

The final modeling population is therefore a subset defined by data availability, not by the target value. Because exclusion rates differ between the initial positive and negative groups, this availability filter is treated as a potential selection-bias threat rather than as evidence that the missingness mechanism is outcome-independent.

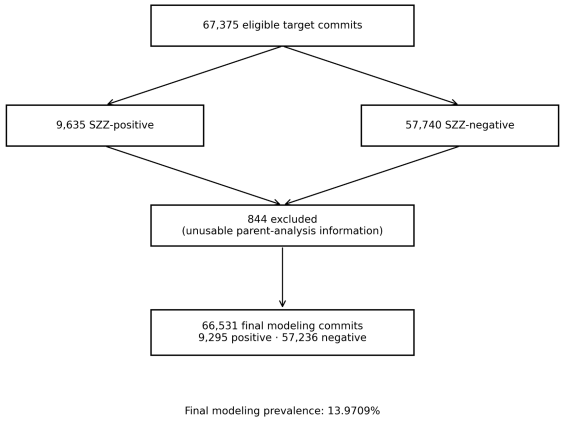


Fig. 1. Construction of the final modeling population from the integrated dataset.

D. Pre-Commit Analysis Alignment

For a target commit C_t, a single parent P_t is selected. The SonarQube analysis associated with P_t supplies the pre-commit state. This prevents post-commit analysis information from being used as a predictor of the target commit.

SonarQube component identifiers do not reliably map one-to-one to Git changed-file paths across the dataset. Consequently, the primary smell representation is constructed at project/analysis level rather than by assuming an exact file-level mapping. This is a deliberate validity choice: an imperfect path mapping could create false file-level smell assignments.

E. Smell Lifecycle Reconstruction

Active smells are reconstructed from SonarQube issue lifecycle information. An issue is active at the selected parent analysis when its creation precedes or coincides with that analysis and it has not been closed before it. The lifecycle audit identified 966,218 CODE_SMELL records: 216,200 had no recorded close analysis, one closed in the same analysis, and 750,017 were created and closed later. No closure-before-creation cases were observed.

An independent validation sample of 100 usable observations matched active-smell totals, active rules, active components, and severity distributions. This provides a direct consistency check on the reconstructed smell state.

IV. FEATURE ENGINEERING

The feature design follows the nested sequence

$$C \rightarrow C+H \rightarrow C+H+\text{Change} \rightarrow C+H+\text{Change}+S. \quad (1)$$

The purpose is to attribute incremental predictive changes to feature groups rather than to compare unrelated model specifications.

A. Conventional Code Features

The conventional code group contains 23 Sonar-derived measures: COMPLEXITY, FILE_COMPLEXITY, CLASS_COMPLEXITY, FUNCTION_COMPLEXITY, COGNITIVE_COMPLEXITY, COVERAGE, LINE_COVERAGE, DUPLICATED_LINES, DUPLICATED_BLOCKS, DUPLICATED_FILES, DUPLICATED_LINES_DENSITY, CLASSES, FILES, DIRECTORIES, FUNCTIONS, STATEMENTS, COMMENT_LINES, COMMENT_LINES_DENSITY, LINES, NCLOC, AFFERENT_COUPLINGS, EFFERENT_COUPLINGS, and NUMBER_OF_CLASSES_AND_INTERFACES.

Features with approximately 99.97% missingness, including COMPLEXITY_IN_CLASSES, COMPLEXITY_IN_FUNCTIONS, and PACKAGE_COUNT, were excluded. Other missing values are handled through training-fitted median imputation.

B. Development History

The history group contains PREVIOUS_COMMITS, COMMITS_LAST_30_DAYS, and COMMITS_LAST_90_DAYS. These variables summarize activity before the target commit. The target commit is excluded from its own historical windows. The corrected construction requires a strictly positive time difference, preventing events exactly at the target timestamp from being treated as historical.

C. Commit Changes

Four current-change features are used: LOG1P_LINES_ADDED, LOG1P_LINES_REMOVED, LOG1P_LINES_CHANGED, and LOG1P_FILES_CHANGED. Each is transformed as $x' = \log(1 + x)$ to reduce the influence of highly skewed commit-size distributions while retaining zero-valued changes.

D. Code Smells

Seven smell features are used: ACTIVE_SMELLS, ACTIVE_SMELL_RULES, ACTIVE_SMELL_COMPONENTS, BLOCKER_SHARE, CRITICAL_SHARE, MAJOR_SHARE, and MINOR_SHARE. Severity shares are calculated as

$$\mathrm{Share}_k = \frac{\mathrm{ActiveSmells}_k}{\mathrm{ActiveSmells}}. \tag{2}$$

When no active smell exists, the share is set to zero. INFO share is not included because the selected severity shares otherwise form a complete partition of the active-smell population.

The final primary representation therefore contains 37 variables: 23 code metrics, three history variables, four change variables, and seven smell variables. The model comparisons add these groups cumulatively rather than comparing arbitrary feature subsets. This is important for interpreting the smell delta: the smell model is evaluated against a baseline that already contains the conventional code, development-history, and current-change information.

Missingness was handled conservatively. Variables with approximately 99.97% missingness, including COMPLEXITY_IN_CLASSES, COMPLEXITY_IN_FUNCTIONS, and PACKAGE_COUNT, were excluded. Other selected variables with substantially lower missingness were retained and median-imputed inside the modeling pipeline. Because the imputation statistics are fitted on the training partition, the procedure does not use future test observations to calculate preprocessing parameters.

An additional design choice concerns recent SZZ fault history. Although recent fault counts were reconstructed during data auditing, they are not included in the primary history group. Such variables are derived from future fault-fixing information and could therefore leak outcome information into a prediction setting. Excluding them keeps the primary history representation restricted to information available before the candidate commit.

V. Experimental Design

A. Prediction Target

The binary target is

$$Y = \begin{cases} 1, & \text{SZZ-derived fault-inducing commit,} \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

The prediction point is defined as the moment when the target commit's current diff is available for assessment, but before any future fault-fixing event is known. Thus, current-change features are available at prediction time, while the smell state is reconstructed from the target's parent analysis. The SZZ-derived label is retrospective: it is assigned from later fixing relationships in the dataset and should not be interpreted as a fully deployment-faithful observable target or as direct causal ground truth.

B. Temporal Holdout

Observations are sorted by COMMITTER_DATE, PROJECT_ID, and COMMIT_HASH. The chronological split contains 46,571 training observations and 19,960 temporally later test observations. The test positive prevalence is 9.80%.

C. Rolling Windows

The rolling experiment is separate from the primary 70/30 holdout. After chronological ordering of all 66,531 modeling rows, ten expanding-training windows are formed with test blocks beginning at 50%, 55%, ..., 95% of the ordered data; each test block contains 3,326 observations. The test blocks are disjoint and correspond to the latter half of the full dataset. Seeds 42, 43, and 44 are repeated within each window for model sensitivity. Because seeds repeat the same temporal windows, they are not treated as independent temporal observations. For reporting, the three seeds are aggregated within each window and the resulting ten window-level deltas are summarized descriptively. The smell delta is

$$\begin{aligned} \Delta S = {} & \mathrm{PR\text{-}AUC}(C + H + \mathrm{Change} + S) \\ & - \mathrm{PR\text{-}AUC}(C + H + \mathrm{Change}). \end{aligned} \tag{4}$$

TABLE III
PRIMARY FEATURE GROUPS USED IN THE NESTED EXPERIMENTS.

Group	Features
C: Code	COMPLEXITY; FILE_COMPLEXITY; CLASS_COMPLEXITY; FUNCTION_COMPLEXITY; COGNITIVE_COMPLEXITY; COVERAGE; LINE_COVERAGE; DUPLICATED_LINES; DUPLICATED_BLOCKS; DUPLICATED_FILES; DUPLICATED_LINES_DENSITY; CLASSES; FILES; DIRECTORIES; FUNCTIONS; STATEMENTS; COMMENT_LINES; COMMENT_LINES_DENSITY; LINES; NCLOC; AFFERENT_COUPLINGS; EFFERENT_COUPLINGS; NUMBER_OF_CLASSES_AND_INTERFACES
H: History	PREVIOUS_COMMITS; COMMITS_LAST_30_DAYS; COMMITS_LAST_90_DAYS
Change	LOG1P_LINES_ADDED; LOG1P_LINES_REMOVED; LOG1P_LINES_CHANGED; LOG1P_FILES_CHANGED
S: Smells	ACTIVE_SMELLS; ACTIVE_SMELL_RULES; ACTIVE_SMELL_COMPONENTS; BLOCKER_SHARE; CRITICAL_SHARE; MAJOR_SHARE; MINOR_SHARE

TABLE IV
CLASSIFIER CONFIGURATIONS USED IN THE PRIMARY EXPERIMENTS.

Model	Configuration
Logistic Regression	max_iter=2000; balanced class weights; standardization after imputation
Random Forest	300 trees; balanced class weights; min_samples_leaf=2; parallel fitting
HistGradientBoosting	300 iterations; learning rate=0.05; max leaf nodes=31; training-instance class weights

TABLE V
TEMPORAL PR-AUC UNDER NESTED FEATURE CONFIGURATIONS.

Model	C	C+H	C+H+Chg.	C+H+Chg.+S
LR	.1365	.1543	.3300	.3387
RF	.2120	.2250	.3758	.3826
HGB	.2059	.2192	.3753	.3847

D. Leave-One-Project-Out

For each project, all observations from that project are excluded from training and the held-out project is evaluated. Three projects have single-class held-out partitions and are therefore excluded from PR-AUC comparison: org.apache:commons-exec, org.apache:hive, and org.apache:ognl. This leaves 28 usable projects. The final LOPO analysis uses seed 42. This is within-dataset leave-one-project-out evaluation, not validation on an independent external dataset.

E. Models and Preprocessing

Logistic Regression, Random Forest, and HistGradientBoosting are evaluated. Logistic Regression uses balanced class weighting and a maximum of 2,000 iterations. Random Forest uses 300 trees, balanced class weighting, and minimum leaf size two. HistGradientBoosting uses 300 iterations, learning rate 0.05, maximum 31 leaf nodes, early stopping disabled, and balanced training weights.

Median imputation is fitted within the training data. Logistic Regression uses standardization after imputation. The preprocessing strategy is applied separately for each training/evaluation split to avoid using future or held-out information.

F. Evaluation and Statistical Analysis

PR-AUC is the primary metric because the positive class is relatively uncommon, and all incremental comparisons reported for the research questions use PR-AUC. PR-AUC is computed with scikit-learn's `average_precision_score`. The rolling analysis is summarized at the ten-window level after averaging the three repeated seeds within each window. No p-value is reported for the rolling experiment because the ten temporal windows, rather than the repeated seeds, are the effective temporal units and the available experiment does not justify treating the seeds as independent observations.

G. Evaluation Protocol Summary

The primary temporal experiment uses one fixed 70/30 chronological partition shared by all four feature configurations within each classifier. Results are averaged over seeds 42, 43, and 44. This paired design means that a change in PR-AUC is measured on the same future observations. The rolling experiment extends this design across ten successive chronological windows and repeats each model with three random seeds. The LOPO experiment changes the generalization axis by holding out complete projects rather than future time periods.

The primary comparison is therefore not a single model score. Instead, the central quantity is the within-evaluation difference between two nested feature configurations. For the smell experiment, this is the difference between C+H+Change+S and C+H+Change. The same future observations are used for adjacent configurations within each evaluation unit, so the comparison isolates the incremental contribution of the added feature group.

VI. RESULTS

A. Overall Temporal Evaluation

The final modeling population contains 66,531 commits, 31 projects, 9,295 positives, and 57,236 negatives. The temporal training and test partitions contain 46,571 and 19,960 observations, respectively.

Adding history to C increases PR-AUC by 0.017791, 0.012997, and 0.013301 for LR, RF, and HGB, respectively. Adding current commit-change characteristics produces much larger increases of 0.175710, 0.150844, and 0.156111 over C+H. Finally, adding smells produces increases of 0.008665, 0.006743, and 0.009464 over C+H+Change.

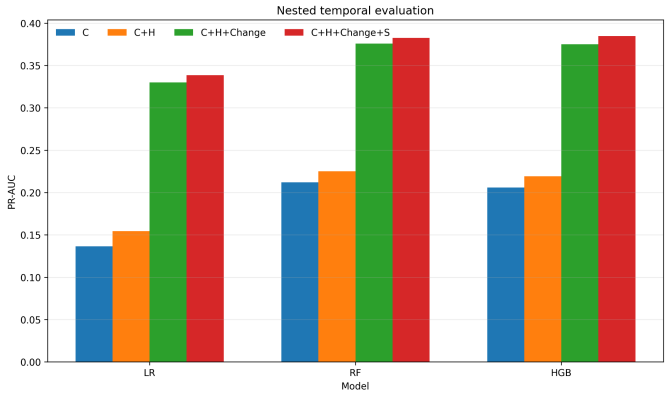


Fig. 2. PR-AUC for the four nested feature configurations under the main chronological evaluation.

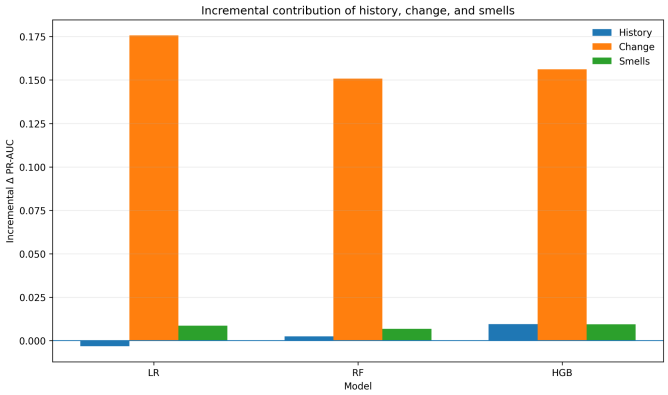


Fig. 3. Incremental PR-AUC contributions of history, current changes, and smells in the main temporal evaluation.

TABLE VI
INCREMENTAL PR-AUC CHANGES IN THE PRIMARY TEMPORAL HOLDOUT.

Model	History	Commit change	Smell
LR	+0.017791	+0.175710	+0.008665
RF	+0.012997	+0.150844	+0.006743
HGB	+0.013301	+0.156111	+0.009464

B. RQ1: Incremental Contribution of Smells

All three classifiers show a positive smell-related change in the main temporal holdout. The contribution is small relative to the contribution of current commit-change characteristics. This supports a narrow RQ1 conclusion: the selected smell representation produces a small positive point-estimate increment in this temporal split. The result is descriptive of this evaluation period and does not by itself establish a statistically robust or causal smell effect.

C. RQ2: Variation Across Temporal Windows

The ten-window experiment is used to examine whether the smell contribution is consistent across successive periods rather than to treat repeated random seeds as independent observations. After averaging seeds within each window, the mean smell deltas are +0.003061 for LR, +0.005259 for RF, and +0.010628 for HGB. The recorded windows include both positive and negative smell deltas, as shown by the retained rolling-window result data. Thus, the average contribution is positive for all three classifiers, but the magnitude varies across time and the experiment does not establish temporal stability as a general property.

The rolling protocol also clarifies why its results are not another partition of the 19,960-row primary test set: the ten 3,326-row test blocks are formed from positions 50% through 100% of the complete 66,531-row ordered population, using expanding training prefixes. This separate evaluation is intended to probe temporal heterogeneity.

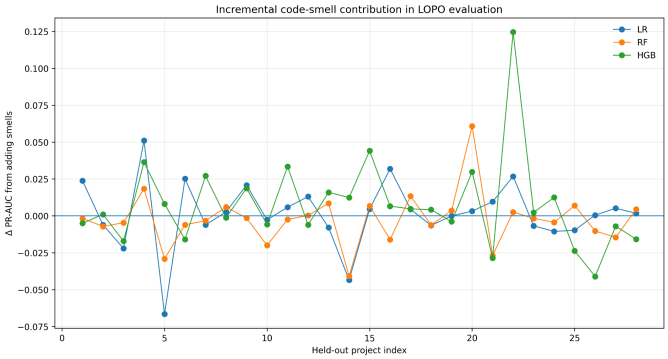


Fig. 4. Project-level smell-related PR-AUC deltas under LOPO evaluation.

D. RQ3: Cross-Project Transfer

Across 28 usable LOPO projects, the mean smell deltas are -0.001821 for LR, -0.003431 for RF, and +0.003778 for HGB. Positive project-level deltas occur in 11/28, 12/28, and 16/28 projects. The signs therefore differ across models, and the project-held-out evidence does not establish a consistent incremental smell contribution.

The signs differ across models and projects, and no consistent cross-project smell contribution is established. Therefore, the positive temporal holdout result should not be generalized as a demonstrated cross-project effect.

E. SZZ Timing and Follow-Up

The SZZ timing audit contains 52,428 links with both inducing and fixing dates. Sixty-seven links have negative fixing-minus-inducing lags; these anomalous links are retained as provided by the dataset and no lag-based filtering is applied. The mean lag is 1,273.84 days, the median is 931.83 days, the 90th percentile is 2,944.91 days, and the 95th percentile is 3,599.91 days. Candidate commits generally have long available follow-up: the median follow-up to the dataset observation end is approximately 4,861 days.

At least 180, 365, and 730 days of follow-up produce essentially the same observed positive rate of approximately 14.30% in the initial 67,375-candidate population (not the final

TABLE VII
LOPO SMELL-RELATED PR-AUC DELTAS ACROSS 28 USABLE PROJECTS.

Model	Mean	Median	SD	Positive
LR	-.001821	-.002762	.014248	11/28
RF	-.003431	-.002596	.021175	12/28
HGB	+.003778	+.003346	.033363	16/28

TABLE VIII
ROLLING AND LOPO SMELL-DELTA SUMMARY.

Model	Rolling mean	LOPO mean	LOPO positive
LR	+.003061	-.001821	11/28
RF	+.005259	-.003431	12/28
HGB	+.010628	+.003778	16/28

66,531-row modeling population). The audit therefore does not support an arbitrary two- or three-year purge of candidates. Right-censoring remains a limitation, but it is documented rather than hidden by an unsupported cutoff.

VII. DISCUSSION

A. Incremental Value Rather Than Simple Association

The central result is not that smells are universally predictive, but that the selected smell representation contributes a small amount of additional predictive information in the primary temporal setting after the other feature groups have been included. This is a stricter question than testing smell features in isolation.

The distinction matters because smell measures overlap conceptually with conventional structural metrics. Complexity, duplication, size, and coupling can already encode some characteristics associated with poorly structured code. Once these predictors are present, the remaining independent information in aggregate smell counts and severity shares can be relatively small.

B. Current Commit Changes Dominate the Incremental Comparison

The strongest empirical contrast is between current-change characteristics and smells. The C+H to C+H+Change improvement ranges from 0.150844 to 0.175710, whereas the subsequent smell improvement ranges from 0.006743 to 0.009464. This does not imply that smells have no relationship with faults. It means that, under this representation and protocol, current-change characteristics account for a much larger incremental PR-AUC change.

This observation also motivates future work on interactions between smells and changes. For example, a smell that is harmless when untouched may have different predictive relevance when a large modification is made to the surrounding code. The current experiment deliberately avoids such interaction terms so that the incremental main-effect contribution can be isolated.

C. Temporal Variation

The rolling analysis demonstrates why a single chronological split is insufficient for making broad claims about stability. The smell effect changes in magnitude and direction across windows. The mean can remain positive while individual windows are negative. Because repeated seeds are not independent temporal units, the revised analysis does not attach inferential p-values to the ten-window summary.

D. Cross-Project Generalization

LOPO results are weaker and more heterogeneous than the main temporal result. Because each held-out project has its own coding conventions, smell distributions, repository history, and class balance, an effect learned from the remaining projects need not transfer unchanged.

The results therefore do not establish that the smell contribution is project-independent. The LOPO analysis is also internal to the same curated dataset, so it should not be interpreted as external validation on an independent population.

E. Comparison With Prior Literature

The present findings should not be interpreted as contradicting earlier smell-aware defect-prediction studies. Prior studies use different datasets, prediction units, feature definitions, labeling mechanisms, and validation protocols [2]–[5]. In particular, the current study asks how much incremental information remains after current commit-change characteristics are included. Differences in experimental protocol can therefore produce different effect magnitudes without implying that one study invalidates another.

F. Interpretation of the Nested Comparison

The nested design provides a more specific interpretation than a conventional smell-versus-no-smell comparison. The final smell model is not compared with a model containing only smells; instead, it is compared with a model that already has access to conventional code measurements, development history, and the current change magnitude. Consequently, the measured smell delta represents information that remains after those baseline sources are available. This also explains why a small delta should not be interpreted as evidence that smells have no relationship with faults. A feature can be associated with the outcome while adding little information after correlated predictors have been included.

G. Temporal Versus Cross-Project Evidence

The temporal and LOPO experiments answer different generalization questions. Temporal evaluation asks whether information learned from earlier observations remains useful for later observations from the same overall project population. LOPO asks whether the incremental contribution remains when an entire project is absent from training. The difference between the two analyses is therefore informative: the positive mean temporal smell deltas coexist with inconsistent project-held-out deltas. The results do not support treating the temporal increment as a universal project-independent effect.

H. Implications for Model Construction

The large incremental change observed when current commit-change characteristics are added indicates that the choice of baseline matters substantially in evaluating smell information. If smells are added to a weak baseline, their apparent contribution can combine with information already represented by other features. The nested approach used here reduces that ambiguity and makes the comparison more directly relevant to whether a smell-aware system adds value beyond information that a JIT prediction system could already obtain from code, history, and the current change.

VIII. Threats to Validity

A. Construct Validity

The target is SZZ-derived rather than manually verified fault causation. SZZ can contain false positives and other labeling errors [6]. The smell representation is project/analysis-level because reliable file-level component mapping is not guaranteed. Thus, the results concern predictive usefulness of the selected pre-commit smell state, not causal effects of individual smells. The commit-level target and project-level smell state also create a granularity mismatch: commits changing very different sets of files can receive the same project/analysis smell representation.

B. Internal Validity

Training-time preprocessing is fitted using training data only. Chronological ordering prevents future predictor observations from entering model fitting, and the nested design uses the same evaluation population for adjacent configurations. Nevertheless, the SZZ target is retrospective and may depend on future fixing activity; chronological predictor splitting does not remove this target-observability limitation. In addition, the 844 availability exclusions remove 3.53% of initial positives versus 0.87% of initial negatives, so selection bias from parent-analysis availability cannot be ruled out. Unmeasured repository, developer, textual, or process characteristics may also contribute predictive information.

C. External Validity

The study uses 31 projects from one curated dataset. Although LOPO provides a project-held-out test, it is not equivalent to validation on a completely independent dataset or industrial population. Results may differ for other repositories, static-analysis tools, programming languages, or development practices.

D. Temporal Validity

The rolling experiment shows temporal variation in smell contribution. The main chronological split should therefore be interpreted as one evaluation period rather than a permanent estimate of future performance. The ten rolling test blocks are separate from the primary 70/30 holdout and use expanding training prefixes.

E. Statistical Validity

The rolling analysis contains ten effective temporal windows per model after aggregating the three repeated seeds within each window. Because the windows are the relevant temporal units, no seed-level pseudoreplication test is reported. LOPO uses 28 usable projects and is summarized at the project level. The absence of a rolling p-value is deliberate: the available evidence supports descriptive heterogeneity analysis rather than an independence assumption across repeated seeds.

IX. Reproducibility and Experimental Traceability

The experiment was implemented as a sequence of auditable data-construction and modeling stages. The retained implementation uses Python with pandas, NumPy, SciPy, and scikit-learn; exact package-version metadata were not preserved in the final experiment log and are therefore a reproducibility limitation. The final figures are generated from measured result data rather than manually specified graphical values. Figure 1 summarizes the dataset construction, Figure 2 visualizes the primary nested temporal PR-AUC results, Figure 4 shows the project-level LOPO deltas, and Figure 3 compares the incremental contributions of the principal feature groups.

For each stage, the paper preserves the distinction between raw database records, the final modeling population, model predictions, and derived statistics. This separation is particularly important for the smell analysis because the target label is obtained through SZZ while the predictor state is reconstructed from a pre-commit SonarQube analysis. The result plots therefore represent a chain of transformations rather than independent hand-curated illustrations.

The empirical workflow was constructed so that each reported figure can be traced to an experimental output. The final modeling dataset is produced from the integrated SQLite database; model runs generate structured result tables; statistical analysis operates on those result tables; and figures are generated from the final result data rather than manually edited numerical values.

The primary temporal figure is derived from the four measured feature configurations for each classifier. The LOPO figure is derived from 28 usable held-out projects. The rolling analysis is retained in the structured result tables and summarized numerically rather than by a figure, avoiding the use of an obsolete seed-level visualization.

This traceability is important because visualizations can otherwise conceal methodological errors. In this study, the main experimental branch was rerun after correcting an earlier history-feature construction problem, and the final results reported here use the corrected feature construction. Obsolete experimental outputs are not used in the reported tables or figures.

X. Conclusion

This study investigated whether pre-commit code-smell information provides incremental predictive value for SZZ-derived fault-inducing commits after accounting for conven-

tional code metrics, development history, and current commit-change characteristics.

Using 66,531 commits from 31 projects, the study evaluated nested feature groups under temporal, rolling-window, and project-held-out validation. The main temporal evaluation showed a small positive smell contribution, while current commit-change characteristics produced substantially larger incremental gains.

The rolling evaluation showed a positive average smell contribution but considerable variation across temporal windows; because repeated seeds were not treated as independent temporal observations, the rolling result is reported descriptively rather than through a seed-level significance test. The LOPO experiment did not establish consistent transfer of the smell contribution to held-out projects within the dataset.

Overall, the evidence indicates that the selected pre-commit smell representation has a small positive point-estimate increment in the evaluated temporal setting, but the contribution is comparatively small, varies across temporal windows, and is not consistently observed in project-held-out evaluation. The study does not establish causality or deployment-level target observability. Future work should investigate finer-grained smell histories, smell-change interactions, alternative smell representations, additional repository predictors, independent datasets, and alternative SZZ-labeling procedures.

ACKNOWLEDGMENT

The authors thank Dr. D. Kothandaraman, School of Computer Science and Engineering (SCOPE), VIT-AP University, for guidance and supervision of this project.

REFERENCES

[1] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A Large-Scale Empirical Study of Just-in-Time Quality Assurance," *IEEE Trans. Software Eng.*, vol. 39, no. 6, pp. 757–773, 2013, doi: 10.1109/TSE.2012.70.

[2] T. Hall, M. Zhang, D. Bowes, and Y. Sun, "Some Code Smells Have a Significant but Small Effect on Faults," *ACM Trans. Software Eng. Methodology*, vol. 23, no. 4, Art. no. 33, 2014, doi: 10.1145/2629648.

[3] F. Palomba, M. Zanoni, F. A. Fontana, A. De Lucia, and R. Oliveto, "Smells Like Teen Spirit: Improving Bug Prediction Performance Using the Intensity of Code Smells," in *Proc. IEEE ICSME*, 2016, pp. 244–255, doi: 10.1109/ICSME.2016.27.

[4] F. Palomba, M. Zanoni, F. A. Fontana, A. De Lucia, and R. Oliveto, "Toward a Smell-Aware Bug Prediction Model," *IEEE Trans. Software Eng.*, vol. 45, no. 2, pp. 194–218, 2019, doi: 10.1109/TSE.2017.2770122.

[5] B. Sotto-Mayor and M. Kalech, "Cross-project Smell-based Defect Prediction," *Soft Computing*, vol. 25, no. 22, pp. 14171–14181, 2021, doi: 10.1007/s00500-021-06254-7.

[6] E. Campos Neto, D. Alencar da Costa, and U. Kulesza, "Revisiting and Improving SZZ Implementations," in *Proc. ACM/IEEE ESEM*, 2019, pp. 1–12, doi: 10.1109/ESEM.2019.8870178.

[7] The Technical Debt Dataset, "The Technical Debt Dataset, Version 2.0," GitHub repository, release 2.0, 2019. [Online]. Available: https://github.com/clowee/The-Technical-Debt-Dataset.

[8] V. Lenarduzzi, N. Saarimaki, and D. Taibi, "The Technical Debt Dataset," in *Proc. PROMISE'19*, 2019, pp. 2–11, doi: 10.1145/3345629.3345630.